AIKDAP — Technical Brief for an IEEE Paper

Source of truth: this repository, as analysed on 2026-09-03. All numbers below are measured from the code/docs, not estimated.

Structure. Part A (§1-§12) is the *engineering* brief: what is built and measured. Part B (§13-§17) is the *product* brief: the three user personas, the stack, the user benefit, the differentiation and the advantages — with an explicit built-vs-planned boundary so the paper never claims a capability the code does not have.

PART A — ENGINEERING BRIEF

1. One-sentence framing

AIKDAP is a multi-agent, project-scoped "AI work operating system" whose central technical contribution is **verifiable non-hallucination**: the guarantee that a generated answer contains no claim unsupported by evidence is enforced by backend control flow and a calibrated relevance gate, not by prompt instructions to the model.

Candidate paper titles

- *Backend-Enforced Grounding: A Three-Stage Retrieval Gate and Citation Validator for Hallucination-Resistant Research Assistants*
- *AIKDAP: A Provenance-Preserving Multi-Agent Architecture for Document- Grounded Research Workflows*

---

2. Problem statement (for Section I)

Existing tools fragment the research workflow: a chat model for reasoning, a search tool for discovery, a notebook tool for documents, a BI tool for analytics. Three concrete gaps follow:

1. **Unverifiable output.** RAG systems prompt the model to "only use the provided context". A prompt is a request, not a guarantee. Nothing structurally prevents a citation to a document that was never retrieved.
2. **Top-k always returns k.** A reranker asked for 5 chunks returns 5, however irrelevant. *Ordering* and *relevance* are distinct questions; conflating them feeds off-topic evidence to the generator and invites fabrication. (Measured in this system: the query "What is the capital of Japan?" retrieved five poultry-industry chunks at cosine similarity ~0.24.)
3. **Provider fragility is silent.** Free-tier LLM quotas fail in ways that look transient but are not; naive retry loops retry a daily quota forever and report only a raw exception string.

---

3. System architecture (Section III)

3.1 Layered view

```
React/TS SPA
   |  REST /api/v1 (JWT)
FastAPI (feature modules: router -> service -> repository -> model)
   |
   |-- Celery workers (Redis broker) - async ingestion + research runs
   |-- LangGraph orchestrator (6 nodes, shared typed state)
   |-- LLM Gateway (single LiteLLM boundary; retry, fallback, breaker)
   \-- Knowledge layer: PostgreSQL 17 + pgvector, Redis, ChromaDB
```

Every module follows the same five-file shape (`router / service / schemas / models / repository`). Business logic never lives in routers. Ownership is enforced transitively through the owning Project, and "exists but not yours" is indistinguishable from "does not exist" at the API boundary.

3.2 The research graph (LangGraph)

```
START -> planner -> router --+--> asset_retrieval --+--> web_research --+
                             |--> web_research -----|                   |
                             +--------------------+-------------------+--> context_builder -> synthesis -> END
```

Six nodes: `planner`, `router`, `asset_retrieval`, `web_research`, `context_builder`, `synthesis`.

Architectural invariants worth stating explicitly in the paper:

- **Agents never call each other.** The only output of a node is a partial update to a shared `ResearchState` (a `TypedDict`); the graph decides who runs next. This is the property that makes the run trace complete.
- **Two state channels use `operator.add` reducers** (`documents`, `messages`) so retrieval results and the agent transcript accumulate; everything else is last-write-wins. Because the reducers already exist, the currently-sequential retrieval chain can be fanned out in parallel without changing the merge semantics.
- **The compiled graph is stateless.** Strategies, DB session and the execution tracker are injected per invocation via `config["configurable"]`, so one compiled instance is shared safely across concurrent runs.
- **Uniform instrumentation.** Every node is wrapped by `tracking.instrument` before registration, which times it, reports start/success/failure to an injected `NodeExecutionTracker`, and applies the registry-declared failure policy. This is necessary because a LangGraph exception surfaces from `ainvoke` with no indication of which node raised it — per-node wrapping is the only place that knowledge exists.
- **Graceful degradation is reported, not hidden.** A non-critical node failure writes a `*_failure` key into `intermediate_results`; `degraded_warnings()` turns those into text the synthesis node must place in the deliverable, so the answer states plainly that a source was unavailable rather than presenting partial evidence as complete.

---

4. Core contribution: the grounding chain (Section IV — the paper's heart)

4.1 Three-stage retrieval

Stage	Model / mechanism	Purpose	Output
0	LLM query reformulation (schema-validated, constraint-preserving)	normalise the question, extract entities/metrics/dates/regions	`search_query` + typed facets
1	**BGE-M3** (1024-d) embeddings via Ollama + **pgvector** cosine	over-retrieve `candidate_k = 20`	ranked candidates + cosine distance
2	**BGE-Reranker-v2-m3** cross-encoder under `llama-server --reranking`	reorder; returns `top_k = 5`	unbounded relevance logit
3	**Relevance gate**, threshold **-2.0**	decide which candidates are relevant *at all*	possibly zero evidence

Stage 1 deliberately over-retrieves: retrieving only `top_k` first would make reranking a no-op. Ownership is enforced *in stage 1*, so the reranker never sees a chunk the caller does not own — there is no path where unrestricted candidates are filtered after scoring.

Scores are never fused or normalised. `retrieval_score` (1 - cosine distance) and `rerank_score` (cross-encoder logit) are measurements on different scales from different models; both are stored side by side and `rerank_score` is *absent*, not zero, when stage 2 did not run. No 0-1 "confidence" is manufactured, because that mapping would be invented rather than measured.

A runtime finding worth a sentence in the paper: the cross-encoder could not be served by Ollama at all (no rerank endpoint exists; `/api/embed` and `/api/generate` crashed the subprocess), while BGE-M3 embeddings and Qwen generation worked on the same instance immediately before and after. The reranker therefore runs under llama.cpp. Because the provider speaks the *standard* rerank contract (`POST {base_url}{path}` with `{model, query, documents}` returning `{results: [{index, relevance_score}]}`), the runtime switch was a configuration change and required no code change.

4.2 The relevance gate is calibrated, not guessed (strongest empirical result)

BGE-Reranker-v2-m3 emits an unbounded logit; observed range on this corpus is roughly [-11, +10], and relevant pairs are frequently negative. Hence 0.0, 0.5, and "positive means relevant" are all meaningless, and the threshold had to be measured.

Method (fully reproducible; raw data in `backend/tests/fixtures_relevance_calibration.json`): the real stack — BGE-M3 -> pgvector -> BGE-Reranker-v2-m3 under llama.cpp, nothing mocked — scored every chunk against every query. Corpus of 7 chunks; 5 in-corpus and 5 out-of-corpus queries; **70 (query, chunk) pairs**, 11 labelled relevant and 59 irrelevant. Labels were assigned from document *content*, never from the score being measured.

Score distribution:

class	n	min	max	mean	median
Relevant	11	-10.759	+10.272	+2.296	+3.262
Irrelevant	59	-11.048	+2.133	-8.156	-10.072

Threshold sweep:

threshold	TP	FN	FP	TN	precision	recall	FPR	FNR
-4.0	9	2	12	47	0.429	0.818	0.203	0.182
-3.0	9	2	8	51	0.529	0.818	0.136	0.182
-2.0	**9**	**2**	**2**	**57**	**0.818**	**0.818**	**0.034**	**0.182**
-1.0	8	3	2	57	0.800	0.727	0.034	0.273
0.0	6	5	2	57	0.750	0.545	0.034	0.455
+2.5	6	5	0	59	1.000	0.545	0.017	0.455

Selected -2.0. Precision rises steeply (0.529 -> 0.818 between -3.0 and -2.0) while recall is still on its maximum plateau. Per-query behaviour is the real justification: all 5 in-corpus queries retain at least one genuinely relevant chunk with no false accepts, and **4 of 5 out-of-corpus queries drop to zero accepted evidence — so no synthesis call is made at all for them.**

An F0.5-optimal threshold exists at +2.25 (precision 1.000, recall 0.545) and was **rejected**: it reduces a question the corpus genuinely answers to zero evidence, converting a correct answer into a false "insufficient evidence". This is a good discussion point — maximising an aggregate metric at the cost of answering real questions is the wrong trade for a research assistant.

Honest limitation to state in the paper: the distributions overlap across [-10.759, +2.133], with 39 of 70 pairs inside the band. This is a *filter*, not a classifier, calibrated to prefer rejecting borderline evidence over admitting it, and it must be recalibrated when the corpus changes materially. The fixture is also small (n=70, 7 chunks) — a scale-up is the obvious reviewer request and the obvious future-work item.

4.3 Backend-enforced grounded synthesis

`GroundedSynthesizer` implements three guarantees in code:

1. **Only retrieved evidence is sent.** The model receives system instructions, the question, the objective, and the evidence blocks — nothing else. No database access, no tools, no conversation history.
2. **Only supplied evidence can be cited.** Every citation identifier the model returns is checked against the identifiers actually placed in its prompt. Unknown identifiers are **dropped and recorded, never repaired** into something plausible.
3. **The model does not grade itself.** `grounding_status` is computed from the *validated* citation set. A model that claims grounding while citing nothing is recorded as `insufficient_evidence`.

`grounding_status` is a first-class, queryable enum column on `research_runs` (not a key inside the citations JSONB), precisely so "show me every run that could not be answered" is a SQL query. It is nullable, because a run that fails before synthesis has no verdict to report and NULL says exactly that.

A second synthesizer, `ExtractiveSynthesizer`, lifts every sentence verbatim from a real evidence item and therefore *cannot* hallucinate. It is the offline path (`SYNTHESIS_GROUNDED=false`) and doubles as a control condition for evaluation.

Evidence sent to the synthesizer is bounded at 12,000 characters; generation runs at temperature 0.2 with a 2048-token ceiling.

4.4 Provenance that survives the whole pipeline

`EvidenceProvenance` travels from chunk to citation: `chunk_id`, `asset_id`, `file_name`, `rank`, `retrieval_rank`, `retrieval_score`, `rerank_score`. A stored citation therefore resolves back to the exact database row the text came from, *and* records how it was selected.

This is enabled at ingestion time by a deliberate chunking decision: `chunk_document` chunks **each extracted unit independently** (page, sheet, slide, section) rather than the concatenated document, so a chunk — and hence a citation — can never straddle a page boundary and become unable to say where it came from. The accepted cost is non-uniform chunk sizes: a short PDF page becomes its own small chunk. Unambiguous provenance was valued above uniform chunk length. Chunk boundaries otherwise snap to paragraph -> sentence -> word breaks within a bounded look-back window (default `chunk_size = 1000`, `chunk_overlap = 100`).

---

5. Document ingestion pipeline (Section III-B)

`extract -> chunk -> AI understanding -> embed`, run once, either automatically after upload or on demand, via one Celery task.

- **Extraction is deterministic.** No model is involved: `pypdf`, `python-docx`, `python-pptx`, `BeautifulSoup`/`lxml`, `openpyxl`. Text is never "recovered" by a language model.
- **OCR (Tesseract via `pytesseract`)** handles scanned pages. A PDF is routed to OCR when the empty-page ratio exceeds 0.5. Tesseract OSD auto-corrects 90/180/270-degree page rotation first — measured live to fix real rotated scans. Confidence floor 40.0, minimum 10 characters, 30 s per-page timeout, <=50 pages, <=40 MP images. A *specific* unusable image yields an empty result with a `skipped_reason`; only a *deployment-wide* condition (OCR disabled, binary absent) raises. A scanned document in a deployment without OCR is reported honestly as unreadable rather than silently empty. Tesseract was chosen over EasyOCR/PaddleOCR (multi-GB PyTorch/Paddle dependencies) and over PyMuPDF (AGPL-3.0 licensing risk) — a defensible engineering-trade-off paragraph.
- **AI document understanding**: local **Qwen 3.5 4B** via Ollama produces a schema-validated summary, keywords, entities and topics. Qwen is *never* asked to recover missing text; empty documents never reach it. Oversized inputs are chunked and merged (6000-char input budget, 1024 output tokens).
- **Best-effort AI, non-fatal by construction.** Both understanding and embedding failures are tracked on their **own** status fields (`AIProfileStatus`, `EmbeddingStatus`) and never alter the asset's `processing_status` or discard already-persisted text/chunks. The deterministic pipeline's success is independent of any model's availability.

---

6. LLM Gateway: resilience as a measured design (Section III-C)

One module imports LiteLLM; a test asserts this containment holds across the whole `app` package. Callers see only `LLMMessage` / `LLMResponse` / a typed `LLMError` hierarchy, so adding a provider changes one file.

The motivating incident (worth quoting in the paper): Gemini's free tier allows 20 `generateContent` requests per project per model per day. The 429 response carries `RetryInfo.retryDelay: "35s"` *on a quota that resets daily*. Any retry loop honouring the provider's own hint retries politely forever and never succeeds. The gateway therefore classifies the quota window rather than trusting the hint:

- **Continuously-refilling windows** (per-minute rate limits) -> retryable, with jittered exponential backoff (base 1.0 s, max 8.0 s, <=2 retries).
- **Day-scale windows** -> never retryable within a run; move straight to the fallback with no retry.
- **Balance exhaustion** -> not retryable either; money does not appear during a backoff.
- **Terminal errors** (bad credentials, malformed request) -> stop the whole chain immediately; never burn a fallback on a request every provider will reject.

`complete()` walks a configured chain — primary -> fallback -> secondary fallback (Gemini A -> Gemini B -> Groq -> OpenRouter in this deployment) — skipping any candidate the health breaker marks exhausted or down, stopping at the first success. Uncredentialed candidates are skipped silently. Only the provider varies across the chain; messages, evidence and every downstream contract are identical, which is the argument for a fallback answer being as trustworthy as a primary one. Per-error-class cooldowns: quota 3600 s, rate limit 60 s, unavailable 60 s, configuration error 300 s.

Distributed breaker state. Provider health lives in Redis (200 ms timeout, 24 h TTL), so the API process and Celery workers share one view instead of each rediscovering the same outage. Retry policy is centralised deliberately: layered retries multiply, and three nested levels of "just three attempts" is twenty-seven calls to a provider that already said no.

Every response carries `attempts`, `fallback_used`, and the model that actually answered — so the paper can report degradation rates, not just success rates.

Secret safety is a stated design goal: the API key is read from a `SecretStr` at call time, never logged, never attached to an exception, never returned; provider error text is scrubbed before reaching a log, because upstream libraries have been observed to echo credentials back in error messages.

---

7. Explainability and auditability (Section V)

Three tables reconstruct any run completely:

- `research_runs` — query, plan (JSONB), objective, final answer, structured citations (JSONB), `grounding_status`, `include_assets` / `include_web` / `max_results` (stored so the run is *reproducible*), `celery_task_id` (correlates to worker logs), `started_at` / `completed_at` / `duration_ms`.
- `research_steps` — one row per node execution, in execution order, with `node_name` stored as a plain string so adding a node needs no migration.
- `agent_messages` — the inter-agent transcript.

A reconciliation worker marks runs stale after 1200 s so a crashed worker never leaves a run pending forever. `task_id` on a run uses `ON DELETE SET NULL`, not CASCADE: archiving a task must not erase the audit trail of work already performed. This makes the *explainability* claim structural rather than aspirational — Section V can show a real reconstructed trace.

---

8. Research Intelligence layer (differentiator vs. generic RAG)

- **Document understanding** (`analysis.py`) — full-document read producing a structured `ResearchDocumentUnderstanding`: equations, variables, metrics, datasets, with each traced to a source location. Explicitly separates *explicitly present* from *inferred*; unstated values must stay null. Logs internal **conflicts** (e.g. Table 1 says 92%, abstract says 89%), classifies **research gaps** against a user goal (REQUIRED / HELPFUL / OPTIONAL / AMBIGUOUS), and rates sufficiency (SUFFICIENT / PARTIALLY_SUFFICIENT / INSUFFICIENT / AMBIGUOUS) with a reason.
- **Cross-paper synthesis** (`cross_paper.py`) — compares a primary against supporting papers under a hard **source boundary**: primary and supporting evidence are never merged, a supporting paper's exact value never fills a primary paper's UNKNOWN, contradictions are reported rather than resolved, and generated hypotheses are forced to be **modal** ("may improve", not "will improve"). Results are validated programmatically against the claimed `paper_id` set — the same "verify, don't trust" pattern as citation validation.
- **Experiment Plan Engine** (`experiment.py`) — turns an equation or a document understanding into a reviewable experiment plan: variables, constraints, variants, parameter sweeps, imported test cases, and visualisation data. Two rules define it:
  - **Deterministic-first.** Equation parsing and constraint derivation use SymPy; numeric work is plain Python; constraint enforcement never defers to the LLM. The LLM is used only where deterministic code cannot help — inferring a variable's research role from prose — and only as the last, lowest-confidence rung of an explicit evidence hierarchy.
  - **Execution boundary.** Nothing in the module runs generated code, trains a model, or produces an observed result. It classifies, validates and organises; the researcher reviews and edits before any execution phase exists. Expression parsing passes through an explicit safety assertion.

---

9. Implementation scale (for the evaluation/implementation section)

Metric	Value
Backend application code	17,792 LOC across 92 Python modules
Backend test code	11,509 LOC, 528 test functions across 34 test modules
Frontend	16,802 LOC across 127 TypeScript/TSX files, 28 test files
Test-to-code ratio (backend)	~0.65:1
REST endpoints	41 across 7 feature modules
Graph nodes	6
DB tables	users, projects, assets, knowledge_chunks, tasks, research_runs, research_steps, agent_messages

Stack: Python 3.12, FastAPI 0.116, SQLAlchemy 2.0 (async), Alembic, PostgreSQL 17 + pgvector 0.5, Redis 8, Celery 5.5, Pydantic v2, LangGraph 0.6.5, LangChain 0.3.27, LiteLLM 1.80, ChromaDB 1.0.20, SymPy 1.13; React + TypeScript + Vite + Tailwind + shadcn/ui + TanStack Query; Docker Compose. Local models: Qwen 3.5 4B (understanding), BGE-M3 1024-d (embedding), BGE-reranker-v2-m3 Q8_0 (reranking) — the entire retrieval and understanding path runs offline with no per-call cost; only synthesis uses a cloud provider.

Notable test classes worth naming: `test_relevance_gate`, `test_grounded_synthesis`, `test_fallback_grounding`, `test_research_citations`, `test_llm_resilience`, `test_docs_exposure`, `test_execution_docker_policy`, `test_experiment_safety`, `test_research_run_reconciliation`.

---

10. Claims you can defend, and limitations you must state

Defensible claims

1. Citation validity is *structurally* guaranteed: an identifier not placed in the prompt cannot appear in a stored citation.
2. The system returns "insufficient evidence" instead of an answer for 4/5 out-of-corpus queries in the calibration fixture, with no synthesis call made at all.
3. Answer provenance resolves to a specific chunk row, with both selection scores retained on their native scales.
4. The pipeline degrades along declared, reported paths: reranker down -> retrieval-only order with explicit `reranking_status`; provider quota exhausted -> fallback chain with `fallback_used` recorded; local model down -> asset text and chunks still persisted.
5. Retrieval, embedding, understanding and OCR run fully locally.

Limitations to state honestly (reviewers will find them otherwise)

- The calibration fixture is small: 70 pairs, 7 chunks, 10 queries, one domain. The threshold is deployment-specific and needs recalibration on corpus change.
- No end-to-end evaluation against a standard benchmark (e.g. a QA or attribution dataset) and no comparison against a baseline RAG system.
- No human evaluation of answer quality; grounding status measures attribution, not correctness or usefulness.
- The relevance gate is a filter, not a classifier; class distributions overlap and 2/11 relevant pairs are rejected at the chosen threshold.
- Retrieval agents run sequentially, not in parallel; latency is not characterised.
- No latency/throughput benchmarks are recorded in the repository.

---

11. Suggested IEEE section map

Section	Draw from
I. Introduction	§2 problem statement; the poultry/Japan example
II. Related Work	RAG, two-stage retrieval, cross-encoder reranking, attribution/citation verification, LLM agent orchestration, circuit breakers
III. System Architecture	§3 layers + graph; §5 ingestion; §6 gateway
IV. Grounding Mechanism	§4 — the paper's core, with both tables from §4.2 as Table I/II
V. Explainability & Reproducibility	§7 three-table trace
VI. Research Intelligence Layer	§8
VII. Implementation & Evaluation	§9 scale table; §4.2 calibration results
VIII. Discussion & Limitations	§10
IX. Conclusion & Future Work	parallel fan-out, larger calibration corpus, benchmark evaluation, experiment execution sandbox

Suggested figures: (1) layered architecture; (2) LangGraph topology; (3) four-stage retrieval funnel with counts 20 -> 5 -> n<=5; (4) the score distribution / threshold-sweep plot from the calibration fixture; (5) the fallback chain state machine; (6) a real reconstructed run trace.

---

12. Where to look in the repo

Topic	File
Graph topology	`backend/app/agents/planner/graph.py`
Shared state contract	`backend/app/agents/planner/state.py`
Per-node tracking / failure policy	`backend/app/agents/planner/tracking.py`
Grounded synthesis + citation validation	`backend/app/agents/planner/synthesis.py`
Query reformulation	`backend/app/agents/planner/reformulation.py`
Document understanding (research)	`backend/app/agents/planner/analysis.py`
Cross-paper synthesis	`backend/app/agents/planner/cross_paper.py`
Experiment Plan Engine	`backend/app/agents/planner/experiment.py`
LLM gateway / resilience	`backend/app/core/llm/gateway.py`, `provider_health.py`
Two-stage retrieval	`backend/app/modules/knowledge_base/service.py`
Reranker client	`backend/app/modules/knowledge_base/reranking.py`
Relevance gate	`backend/app/modules/knowledge_base/relevance.py`
Chunking + provenance	`backend/app/modules/assets/processing/chunker.py`
Extraction / OCR	`backend/app/modules/assets/processing/extractors.py`, `ocr.py`
Ingestion orchestration	`backend/app/modules/assets/processing/pipeline.py`
Run persistence schema	`backend/app/modules/research/models.py`
Calibration write-up	`docs/relevance-calibration.md`
Resilience write-up	`docs/provider-resilience.md`
Calibration raw data	`backend/tests/fixtures_relevance_calibration.json`

---

13. The product thesis and the three personas

13.1 Thesis

A research paper is not the end of a workflow — it is the *input* to one. What a reader must do next differs completely by who they are, yet every existing tool hands all of them the same thing: a chat window and a summary.

AIKDAP's product claim is that **one ingested, verified, provenance-preserving representation of a document can serve three fundamentally different downstream workflows**, so a user never leaves the platform to finish their work. The expensive, quality-determining part — extract, OCR, chunk with provenance, embed, understand, retrieve, gate, ground — is done **once, per project**. The three personas are three consumers of that same substrate, not three products.

This is the framing that makes the multi-persona design a *technical* argument rather than a feature list: the personas share the grounding guarantees of Part A, and each inherits them for free.

13.2 Persona 1 — The Researcher

Goal: continue an actual line of research, not read a summary of it.

Workflow:

1. Upload reference papers into a project.
2. AIKDAP analyses each into a structured `ResearchDocumentUnderstanding` — equations, variables, metrics, datasets, each traced to a source location.
3. The system judges the corpus **against the researcher's stated goal** and returns a verdict: SUFFICIENT / PARTIALLY_SUFFICIENT / INSUFFICIENT / AMBIGUOUS, with a reason, plus a classified list of what is *missing* (REQUIRED / HELPFUL / OPTIONAL / AMBIGUOUS).
4. If the corpus is insufficient, the platform asks the researcher to upload more — and recommends relevant papers they have not uploaded.
5. Where the research revolves around equations, the platform surfaces the equation and its mutable variables, and visualises — beside it — how changing a variable moves the system's behaviour.
6. Cross-paper comparison reconciles multiple papers without ever merging their evidence, reporting contradictions rather than resolving them.

Status: steps 1-3 and 6 are **built**. Step 4's *ask for more* is built (the gap classification drives it); the *recommend papers you didn't upload* half is **not built** — there is no arXiv / Semantic Scholar integration, and the web research node is currently a deterministic mock that marks every result `simulated: true` behind a `mock://` URI scheme. Step 5 is **partly built**: the Experiment Plan Engine extracts equations, variables and constraints with SymPy and plots an output against an input, but it plots *supplied test cases* and does not yet evaluate the equation live as a slider moves. See §16 for why that last gap is deliberate, not an oversight.

13.3 Persona 2 — The Student

Goal: turn a paper into the artefacts coursework actually requires.

Workflow: upload a paper, receive a synopsis, a slide deck, a report, and similar deliverables — each generated only from what the paper actually says, and each carrying citations back to the page it came from.

Status: the substrate is built — extraction (PDF/DOCX/PPTX/XLSX/HTML), OCR for scanned papers, page-accurate chunking, document understanding, and grounded synthesis with validated citations. The **artefact generators themselves are not built**: there is no synopsis, slide-deck, or report export module in the codebase today. (`python-pptx` appears in the stack as an *input* extractor for uploaded decks, not as an output generator.)

This is the cheapest persona to complete, because a deliverable generator is a new consumer of `ResearchDocumentUnderstanding` plus grounded synthesis — no new retrieval, no new grounding machinery.

13.4 Persona 3 — The Project Builder

Goal: implement the thing a paper describes.

Workflow: upload a paper, state an intent to build, and the platform guides implementation and recommends what would work better for that specific project.

Status: not built. No scaffolding, roadmap, or recommendation module exists. The foundations that a build-guidance agent would need, however, are in place and are the hard parts: structured extraction of methods, datasets and hyperparameters; the sufficiency verdict (a paper that omits its learning rate cannot be reimplemented, and AIKDAP already detects and says so); the conflict log; and the Experiment Plan Engine's variable/constraint model.

13.5 How to present this in the paper — important

Do **not** write Personas 2 and 3 in the present tense. Reviewers check.

The honest and *stronger* framing is architectural: position the three personas as **evidence that the contribution generalises**. The paper's core claim is a grounded, provenance-preserving document substrate; the personas demonstrate that the same substrate serves synthesis, artefact generation, and implementation guidance. State plainly that Persona 1 is implemented and evaluated, and that Personas 2 and 3 are designed on the same substrate and identified as future work. A paper with one deeply-validated persona and a credible generalisation argument is far more publishable than one claiming three shallow ones.

Suggested sentence for the abstract: *"We implement and evaluate the researcher workflow end to end, and show that the same grounded substrate admits two further workflows — student artefact generation and implementation guidance — without modification to the retrieval or grounding layers."*

---

14. What we used to build the platform

14.1 Backend

Concern	Choice	Why it was chosen
API	FastAPI 0.116, Python 3.12	native async; Pydantic v2 request/response validation is the same type system used everywhere else
ORM / migrations	SQLAlchemy 2.0 (async), Alembic 1.16	typed `Mapped[]` models; schema changes only ever via migration
Relational + vector store	PostgreSQL 17 + **pgvector 0.5**	one database for rows *and* embeddings — ownership filters and vector search execute in a single query, so the reranker can never see a chunk the caller does not own
Cache / broker / breaker state	Redis 8	Celery broker, plus shared LLM provider-health state so API and workers see one view of an outage
Background execution	Celery 5.5	ingestion and research runs are long-running; the API returns `202` and the run is tracked
Agent orchestration	**LangGraph 0.6.5** + LangChain 0.3.27	explicit graph topology with a typed shared state and reducers — agents never call each other
Model routing	**LiteLLM 1.80**	one provider-agnostic call surface; contained in a single module, enforced by a test
Deterministic math	**SymPy 1.13**	equation parsing and constraint derivation must not go through an LLM
Vector store (secondary)	ChromaDB 1.0.20	present in the stack alongside pgvector
Logging	structlog 25.4	structured events, so a run is queryable in logs as well as in the database
Quality	pytest, ruff, black, isort, mypy	528 tests; static typing throughout

14.2 Document ingestion

`pypdf` (PDF), `python-docx` (DOCX), `python-pptx` (PPTX), `openpyxl` (XLSX), `BeautifulSoup` + `lxml` (HTML) — all deterministic, no model involved. **Tesseract** via `pytesseract` + `Pillow` for scanned pages, with OSD rotation correction.

Tesseract was chosen over EasyOCR and PaddleOCR (each pulls multi-gigabyte PyTorch/PaddlePaddle GPU stacks) and over PyMuPDF (AGPL-3.0 dual licence — a real risk for a project that may become closed-source). Tesseract is ~19 MB, Apache-2.0, fully offline, no API key, no per-call cost.

14.3 AI models

Role	Model	Where it runs
Document understanding	**Qwen 3.5 4B**	local, via Ollama
Embedding	**BGE-M3**, 1024-d	local, via Ollama
Reranking	**BGE-Reranker-v2-m3** (Q8_0 GGUF)	local, via `llama-server --reranking`
Planning + synthesis	Gemini, with Groq and OpenRouter as fallbacks	cloud, via the gateway

Only synthesis leaves the machine. Ingestion, understanding, embedding, retrieval and reranking are entirely local — which is simultaneously a cost argument, a privacy argument, and an availability argument.

14.4 Frontend

React + TypeScript + Vite, Tailwind CSS, shadcn/ui, TanStack Query — 16,802 LOC across 127 files with 28 test files. The UI is built around a Command Center that answers *"what is happening?"* rather than *"what do you want to ask?"*, plus per-feature views: upload and processing timeline, research pipeline and evidence funnel, citation list and evidence drawer, document and cross-paper analysis panels, and the experiment playground with its visualisation chart.

14.5 Infrastructure

Docker + Docker Compose (`pgvector/pgvector:pg17`, `redis:8-alpine`, backend, worker), JWT authentication (HS256, 30-minute access / 7-day refresh), secrets only via environment, and a startup validation pass that refuses to boot on an unsafe configuration.

---

15. How it helps users

Everyone, on every persona:

- **One workspace, one upload.** The project is the unit of memory, not the chat. Assets, chunks, runs, citations and experiment plans persist and accumulate; nothing has to be re-explained or re-uploaded, and no work disappears when a conversation ends.
- **Answers you can check.** Every claim carries a citation that resolves to a specific page of a specific uploaded file, with the scores that caused it to be selected. Verification is a click, not a re-read.
- **It says "I don't know."** When the evidence does not clear the calibrated relevance threshold, the system reports insufficient evidence and makes no generation call at all — the failure mode is silence, not a confident fabrication.
- **It keeps working when things break.** A dead reranker degrades to retrieval-only order and says so; an exhausted provider quota moves to the next provider and records that it did; a failed local model still leaves the document's text and chunks safely extracted.

Researcher: the sufficiency verdict answers the question that actually blocks progress — *do I have enough to proceed?* — and the gap list says precisely what is missing and how badly it is needed. Cross-paper comparison surfaces contradictions between papers instead of averaging them into a bland consensus, and hypotheses are forced into modal language so a suggestion is never mistaken for a finding. The equation and variable model turns a static paper into something inspectable.

Student: the same verified substrate underlies every artefact, so a synopsis, a deck and a report are three views of one checked understanding rather than three independent opportunities to hallucinate — and each stays traceable to a page number a supervisor can verify.

Project builder: the platform detects the specific omission that makes a paper unimplementable — a missing learning rate, an unstated dataset size, a table that contradicts the abstract — before the user wastes a week discovering it in code.

---

16. How it differs from existing tools

	ChatGPT	Perplexity	NotebookLM	Elicit / SciSpace	AIKDAP
Grounded in *your* documents	partial	no	yes	yes	**yes**
Citation validity enforced in code	no	no	no	no	**yes**
Can return *zero* evidence rather than answer	rarely	no	rarely	rarely	**yes, calibrated**
Provenance to page + selection scores	no	link only	passage	passage	**chunk id + both raw scores**
Full execution trace persisted	no	no	no	no	**yes, 3 tables**
Corpus sufficiency verdict vs. a stated goal	no	no	no	partial	**yes, 4-way + gap list**
Retrieval/embedding runs locally	no	no	no	no	**yes**
Provider-outage resilience visible to the user	n/a	no	no	no	**yes**
Unit of memory	chat	query	notebook	query	**project**

Five differentiators, in the order they matter for a paper:

1. **Grounding is enforced, not requested.** Every competitor asks the model to stay grounded in a prompt. AIKDAP validates the returned citation identifiers against the identifiers actually placed in the prompt, drops unknown ones without repair, and computes `grounding_status` in the backend — the model does not grade itself. This is the difference between a policy and a guarantee.
2. **Relevance is a measured decision, not a top-k side effect.** The gate is calibrated on real (query, chunk) pairs from the deployment's own corpus (precision 0.818 / recall 0.818 at -2.0), and it drops 4 of 5 out-of-corpus queries to zero evidence. No competitor separates *ordering* from *relevance*; asked for five results, they all return five.
3. **Explainability is structural.** `research_runs` / `research_steps` / `agent_messages` reconstruct any run node by node, with timings, the plan, the inputs that make it reproducible, and the Celery task id linking it to worker logs. Explainability here is a schema, not a generated narration of what the model claims it did.
4. **Honest degradation.** Simulated web evidence is flagged `simulated` and carries a `mock://` scheme so it can never be mistaken for real; a missing rerank score is *absent*, not zero; a non-critical agent failure becomes a warning inside the deliverable. The system's design principle is that it would rather report less than imply more.
5. **Local-first economics.** Everything except synthesis runs offline with no per-call cost, which is what makes an over-retrieve-then-rerank-then-gate pipeline affordable at all — three model passes per query would be prohibitive against a metered API.

And the product-level difference: the competitors are all *single-persona*. AIKDAP's substrate is designed so the researcher, the student and the project builder consume one verified understanding rather than three separate tools each re-reading the same PDF.

---

17. Advantages of the platform

Technical

1. Hallucination resistance is verifiable by construction — an identifier never placed in the prompt cannot appear in a stored citation.
2. Zero-evidence is a first-class, calibrated outcome rather than a failure.
3. Provenance survives the entire pipeline, from extracted page to stored citation, with both selection scores on their native scales.

4. Every run is fully reconstructable from three tables.
5. Failure is contained and declared at every layer: reranker, provider, local model, OCR, and per-node agent failure each have a defined, *reported* degraded path.
6. Local-first: retrieval, embedding, understanding and OCR need no API key, incur no per-call cost, and keep documents on the user's infrastructure.
7. Deterministic where determinism is possible — extraction, chunking, equation parsing and constraint enforcement never route through an LLM.

Architectural

8. One LiteLLM boundary, enforced by a test, so adding a provider changes one file.
9. Uniform module shape (`router / service / schemas / models / repository`) across all seven feature modules.
10. A stateless compiled graph with injected dependencies, safe to share across concurrent runs, and reducers already in place for future parallel fan-out.
11. 528 backend tests at a ~0.65:1 test-to-code ratio, including tests that guard *architectural* properties (LiteLLM containment, docs exposure, Docker execution policy, experiment safety) and not merely behaviour.
12. Node names persist as plain strings, so extending the graph never requires a database migration.

Product

13. Project-scoped memory: work accumulates instead of evaporating with a conversation.
14. One substrate, three personas — new deliverables are new consumers of an existing verified understanding.
15. Safety by boundary: the Experiment Plan Engine deliberately never runs generated code, trains a model, or reports an observed result. It classifies, validates and organises, and the researcher reviews before anything executes. **State this as a design decision in the paper, not a limitation** — an automated system that silently fabricates an "experimental result" is precisely the failure mode a grounding paper exists to prevent.